

项目 10 简单个人图书管理系统

一、问题描述

学生在学习过程中拥有很多书籍,对自己购买的书籍进行分类和统计是一种良好的习惯。如果用文件来存储书号、书名、作者名、价格与购买日期等相关书籍的信息,辅之以程序对里面的书籍信息进行统计和查询,将使管理工作变得轻松而有趣。

二、基本要求

(1) 系统至少应具备如下的功能:

- ① 存储书籍各种相关信息;
- ② 提供查找功能,按照书名或作者名查找需要的书籍;
- ③ 提供插入、删除与更新功能;
- ④ 排序功能,按照作者名对所有的书籍进行排序,并按排序后的结果进行显示。

(2) 要求程序能按书号、书名索引。

三、工具及准备工作

在开始实验前,应回顾或复习相关的内容。

需要一台计算机,其中安装有 Visual C++ 6.0、Visual C++ 2017、Dev-C++ 或 CodeBlocks 等集成开发环境软件。

四、分析与实现

为具有实用性,采用文件的形式存储书籍信息。需要操作时,从文件中读入相关记录。由于要求按书号、书名索引,书号为关键字,书名为次关键字,采用多重表文件方式组织索引。为接收文件中的内容,需要有结构来存储相应内容,并且建立索引,还应建立相应的索引项结构。在查找和排序时需要记录或关键字进行比较操作,为此重载相关关系运算符。结构声明与重载的运算符如下:

```
// 日期
struct DateType
{
    int year, month, day;           // 年、月和日
};

// 图书结构
struct BookType
{
    char num[14];                   // 书号
    char title[21];                 // 书名
    long titleNext;                 // 书名相同的下一本书
    char author[11];                // 作者名
    float price;                    // 价格
};
```

```

        DateType buyDate;                // 购买日期
        bool isDeleted;                  // 删除标记
    };

    // 重载排序与查找需要的关系运算符
    bool operator <=(const BookType &first, const BookType &second); // 重载关系运算符<=
    bool operator >=(const BookType &first, const BookType &second); // 重载关系运算符>=

    // 书号索引项结构
    struct NumIndexItemType
    {
        char num[14];                    // 书号
        long offset;                     // 对应记录在主文件中的位置
    };

    // 重载排序与查找需要的关系运算符
    bool operator <=(const char num[14], const NumIndexItemType &item);
                                                // 重载关系运算符<=
    bool operator <(const char num[14], const NumIndexItemType &item);
                                                // 重载关系运算符<
    bool operator ==(const char num[14], const NumIndexItemType &item);
                                                // 重载关系运算符==

    // 书名次索引项结构
    struct TitleIndexItemType
    {
        char title[21];                  // 书名
        long head;                       // 头指针
    };

    // 重载排序与查找需要的关系运算符
    bool operator <=(const char title[21], const TitleIndexItemType &item);
                                                // 重载关系运算符<=
    bool operator <(const char title[21], const TitleIndexItemType &item);
                                                // 重载关系运算符<
    bool operator ==(const char title[21], const TitleIndexItemType &item);
                                                // 重载关系运算符==

```

将对数据文件的相关操作及相关数据封装成个人图书管理系统类 MyBookManageSystem。具体声明及相关常量定义如下：

```

static const long INCREMENT_OF_INDEX_TABLE = 100;    // 索引表增量

// 个人图书管理系统类
class MyBookManageSystem
{

```

```

private:
// 数据成员
    fstream bookFile; // 图书文件
    NumIndexItemType * pNumIndexTable; // 书号主索引表
    long sizeofNumIndex; // 书号主索引表当前索引项数
    long maxSizeOfNumIndex; // 书号主索引表最大索引项数
    TitleIndexItemType * pTitleIndexTable; // 书名次索引表
    long sizeofTitleIndex; // 书名次索引表当前索引项数
    long maxSizeOfTitleIndex; // 书名次索引表最大索引项数

// 辅助函数
    void Display(const BookType &book) const; // 显示图书信息
    void InsertHelp(BookType &book, long offset); // 插入记录辅助函数
    void Insert(); // 插入记录
    void DeleteHelp(BookType &book, long offset); // 删除记录辅助函数
    void Delete(); // 删除记录
    void Update(); // 更新记录
    void SearchByTitle(); // 按书名查询记录
    void SearchByAuthor(); // 按作者名查询记录
    void Search(); // 查询记录
    void SortByAuthor(); // 按作者名排序

public:
// 构造函数, 析构函数与方法
    MyBookManageSystem(); // 无参数的构造函数
    virtual ~MyBookManageSystem(); // 析构函数
    void Run(); // 处理图书
};

```

由于插入与删除操作都要对索引表做相应操作,为简单起见,对更新操作,更新索引表时,先在索引表中删除旧索引项,再插入新索引项,为此定义辅助函数 InsertHelp()与 DeleteHelp()完成相应功能。具体实现如下:

```

void MyBookManageSystem::InsertHelp(BookType &book, long offset)
// 操作结果: 插入辅助函数, book 为要插入的记录, offset 为记录在主文件中的插入位置
{
    if (sizeofNumIndex >= maxSizeOfNumIndex)
    { // 书号主索引项已达到最大容量
        maxSizeOfNumIndex += INCREMENT_OF_INDEX_TABLE; // 扩大最大容量
        NumIndexItemType * pTem; // 临时索引表
        pTem = new NumIndexItemType[maxSizeOfNumIndex]; // 重新分配存储空间
        for (int pos = 0; pos < maxSizeOfNumIndex; pos++)
            pTem[pos] = pNumIndexTable[pos]; // 复制索引项
        delete [] pNumIndexTable; // 释放 pNumIndexTable
        pNumIndexTable = pTem; // pNumIndexTable 指向新存储空间
    }
}

```

```

int j; // 临时变量
for (j = sizeofNumIndex - 1; j >= 0 && book.num < pNumIndexTable[j]; j--)
{
    // 将比 book.num 大的书号主索引项后移
    pNumIndexTable[j + 1] = pNumIndexTable[j];
}
strcpy(pNumIndexTable[j + 1].num, book.num); // 学号
pNumIndexTable[j + 1].offset = offset; // 记录在主文件中的相应位置
sizeofNumIndex++; // 书号主索引项个数自加 1

if (sizeofTitleIndex >= maxSizeOfTitleIndex)
{
    // 书名次索引项已达到最大容量
    maxSizeOfTitleIndex += INCREMENT_OF_INDEX_TABLE; // 扩大最大容量
    TitleIndexItemType * pTem; // 临时索引表
    pTem = new TitleIndexItemType[maxSizeOfTitleIndex]; // 重新分配存储空间
    for (int pos = 0; pos < sizeofTitleIndex; pos++)
        pTem[pos] = pTitleIndexTable[pos]; // 复制索引项
    delete [] pTitleIndexTable; // 释放 pNumIndexTable
    pTitleIndexTable = pTem; // pNumIndexTable 指向新存储空间
}
int pos = BinSearch(pTitleIndexTable, sizeofTitleIndex, book.title);
// 二分查找

if (pos == -1)
{
    // 查找失败
    for (j = sizeofTitleIndex - 1; j >= 0 && book.title < pTitleIndexTable[j]; j--)
    {
        // 将比 book.title 大的书名次索引项后移
        pTitleIndexTable[j + 1] = pTitleIndexTable[j];
    }
    strcpy(pTitleIndexTable[j + 1].title, book.title); // 书名
    pTitleIndexTable[j + 1].head = offset; // 记录在主文件中的相应位置
    sizeofTitleIndex++; // 书名次索引项个数自加 1
    book.titleNext = -1; // 书名链表只有一个节点, 无后继
}
else
{
    // 查找成功
    book.titleNext = pTitleIndexTable[pos].head;
    // book 后继为 pTitleIndexTable[pos].head
    pTitleIndexTable[pos].head = offset; // book 为书名链表第 1 节点
}
}

void MyBookManageSystem::DeleteHelp(BookType &book, long offset)
// 操作结果: 删除记录辅助函数
{
    int pos = BinSearch(pNumIndexTable, sizeofNumIndex, book.num); // 二分查找

```

```

    for (int i = pos + 1; i < sizeofNumIndex; i++)
    {
        // 在书号主索引表中删除索引项
        pNumIndexTable[i - 1] = pNumIndexTable[i];
    }
    sizeofNumIndex--; // 书号主索引项个数自减 1

    pos = BinSearch(pTitleIndexTable, sizeofTitleIndex, book.title); // 二分查找
    long pre = -1, p = pTitleIndexTable[pos].head; // p 指向当前记录, pre 指向前驱记录
    BookType tem; // 临时变量
    while (p != offset)
    {
        // pre 与 p 移向后继
        pre = p; // p 为 pre 的后继
        bookFile.clear(); // 清除标志
        bookFile.seekg(p, ios::beg); // 文件定位
        bookFile.read((char *)&tem, sizeof(BookType)); // 读主文件
        p = tem.titleNext; // p 移向后继
    }
    if (pre == -1)
    {
        // book 为链表的第 1 个节点
        pTitleIndexTable[pos].head = book.titleNext; // book 后继为第 1 节点
    }
    else
    {
        // book 不为书名链表的第 1 个节点
        bookFile.clear(); // 清除标志
        bookFile.seekg(pre, ios::beg); // 文件定位
        bookFile.read((char *)&tem, sizeof(BookType)); // 读主文件
        tem.titleNext = book.titleNext; // pre 后继改为 book 的后继
        bookFile.clear(); // 清除标志
        bookFile.seekg(pre, ios::beg); // 文件定位
        bookFile.write((char *)&tem, sizeof(BookType)); // 写主文件
    }
}

```

按作者名排序时,先将数据主文件记录读入内存,在内存中对记录进行排序,然后再将排序后的记录重新写回数据主文件,并同时建立索引文件。具体实现如下:

```

void MyBookManageSystem::SortByAuthor()
// 操作结果: 按作者名排序
{
    BookType * books = new BookType[sizeofNumIndex]; // 图书数组
    BookType book; // 图书记录
    bookFile.clear(); // 清除标志
    bookFile.seekg(0, ios::end); // 定位到文件尾
    long endOffset = bookFile.tellg(); // 文件尾位置
    bookFile.clear(); // 清除标志
    bookFile.seekg(0, ios::beg); // 定位到文件头
}

```

```

int n=0; // 当前读到的未被删除的记录数
do
{
    bookFile.read((char *)&book, sizeof(BookType)); // 读主文件
    if (!book.isDeleted)
    { // 将未被删除的记录存入 books
        books[n++] = book;
    }
}
while (endOffset != bookFile.tellg());
QuickSort(books, n); // 快速排序
bookFile.clear(); // 清除标志
bookFile.seekg(0, ios::beg); // 定位到文件头

sizeOfNumIndex = sizeOfTitleIndex = 0; // 清空索引表
cout << setw(14) << "书号" << setw(21) << "书名" << setw(11) << "作者名" <<
    setw(8) << "价格" << setw(12) << "购买日期" << endl;
for (int pos = 0; pos < n; pos++)
{ // 依次将 books[] 中的记录写入 bookFile

    long offset = bookFile.tellg(); // 记录插入位置
    InsertHelp(books[pos], offset); // 在索引表中插入 books[pos]
    bookFile.write((char *)&books[pos], sizeof(BookType)); // 写主文件
    Display(books[pos]); // 显示图书信息
}
}

```

按书名查找时,由于建立有索引,可先在索引表中进行查询,找到后,根据索引表中 head 项在主文件的书名链表中查找相应记录,具体实现如下:

```

void MyBookManageSystem::SearchByTitle()
// 操作结果:按书名查询记录
{
    char title[21]; // 书名
    cout << "输入书名: ";
    cin >> title;
    int pos = BinSearch(pTitleIndexTable, sizeOfTitleIndex, title); // 二分查找
    if (pos == -1)
    { // 查找失败
        cout << "查无此书!" << endl;
    }
    else
    { // 查找成功
        BookType book; // 图书记录
        cout << setw(14) << "书号" << setw(21) << "书名" << setw(11) << "作者名" <<

```

```

        setw(8) << "价格" << setw(12) << "购买日期" << endl;
        long offset = pTitleIndexTable[pos].head;           // 记录在主文件中的位置
        while (offset != -1)
        {
            bookFile.clear();                               // 清除标志
            bookFile.seekg(offset, ios::beg);               // 定位文件
            bookFile.read((char *)&book, sizeof(BookType)); // 读主文件
            Display(book);                                   // 显示图书信息
            offset = book.titleNext;                         // 下一同名图书
        }
    }
}

```

按作者名查询记录时,由于没有相应的索引表,只能依次读出图书记录,按顺序进行查找。具体实现如下:

```

void MyBookManageSystem::SearchByAuthor()
// 操作结果:按作者名查询记录
{
    char author[11];                                       // 作者名
    cout << "输入作者名: ";
    cin >> author;
    BookType book;                                         // 图书记录
    bookFile.clear();                                     // 清除标志
    bookFile.seekg(0, ios::end);                           // 定位到文件尾
    long endOffset = bookFile.tellg();                     // 文件尾位置
    cout << setw(14) << "书号" << setw(21) << "书名" << setw(11) << "作者名" <<
        setw(8) << "价格" << setw(12) << "购买日期" << endl;
    bookFile.clear();                                     // 清除标志
    bookFile.seekg(0, ios::beg);                           // 定位到文件头
    do
    {
        bookFile.read((char *)&book, sizeof(BookType)); // 读主文件
        if (strcmp(book.author, author) == 0) Display(book); // 显示图书信息
    }
    while (endOffset != bookFile.tellg());
}

```

五、测试与结论

测试时,应注意尽量覆盖算法的各种情况,屏幕显示参考如下:

1.插入记录 2.删除记录 3.更新记录 4.查找记录 5.按作者名排序 6.退出

输入选择: 1

输入书号: 9787302023685

输入书名: 数据结构

输入作者名: 严蔚敏

输入价格: 22



输入购买日期: 2003 12 18

插入成功!

1.插入记录 2.删除记录 3.更新记录 4.查找记录 5.按作者名排序 6.退出

输入选择: 1

输入书号: 9787811240351

输入书名: OpenCV 教程

输入作者名: 刘瑞祯

输入价格: 49

输入购买日期: 2008 6 19

插入成功!

1.插入记录 2.删除记录 3.更新记录 4.查找记录 5.按作者名排序 6.退出

输入选择: 5

书号	书名	作者名	价格	购买日期
9787811240351	OpenCV 教程	刘瑞祯	49	2008-6-19
9787302023685	数据结构	严蔚敏	22	2003-12-18

1.插入记录 2.删除记录 3.更新记录 4.查找记录 5.按作者名排序 6.退出

输入选择: 4

1.按照书名查找 2.按照作者名查找 3.退出查找

输入选择: 1

输入书名: 数据结构

书号	书名	作者名	价格	购买日期
9787302023685	数据结构	严蔚敏	22	2003-12-18

1.按照书名查找 2.按照作者名查找 3.退出查找

输入选择: 2

输入作者名: 刘蔚敏

书号	书名	作者名	价格	购买日期
----	----	-----	----	------

1.按照书名查找 2.按照作者名查找 3.退出查找

输入选择: 3

1.插入记录 2.删除记录 3.更新记录 4.查找记录 5.按作者名排序 6.退出

输入选择: 3

输入书号: 9787302023685

书号	书名	作者名	价格	购买日期
9787302023685	数据结构	严蔚敏	22	2003-12-18

输入书号: 9787302023685

输入书名: 数据结构

输入作者名: 严蔚敏

输入价格: 22

输入购买日期: 2005 8 16

更新成功!

1.插入记录 2.删除记录 3.更新记录 4.查找记录 5.按作者名排序 6.退出

输入选择: 5

书号	书名	作者名	价格	购买日期
9787811240351	OpenCV 教程	刘瑞祯	49	2008-6-19
9787302023685	数据结构	严蔚敏	22	2005-8-16

1.插入记录 2.删除记录 3.更新记录 4.查找记录 5.按作者名排序 6.退出

输入选择: 2

输入书号: 9787811240351

书号	书名	作者名	价格	购买日期
9787811240351	OpenCV 教程	刘瑞祯	49	2008-6-19

删除成功!

1.插入记录 2.删除记录 3.更新记录 4.查找记录 5.按作者名排序 6.退出

输入选择: 5

书号	书名	作者名	价格	购买日期
9787302023685	数据结构	严蔚敏	22	2005-8-16

1.插入记录 2.删除记录 3.更新记录 4.查找记录 5.按作者名排序 6.退出

输入选择:

从上面的显示结果来看,程序满足基本要求。

六、思考与感悟

程序采用多重表文件来解决多关键字文件,读者还可采用倒排文件来实现;由于删除操作只做删除标志,没有从物理上删除,还可设计重构数据操作,用以从物理上删除记录。

本系统的实现包含了查找、排序与文件操作,覆盖知识面广,只有多做这些综合性的项目,才能迅速提高算法设计能力,为应用所学知识解决实际问题打下坚实的基础。